Week 6 APP DEV-2

2024 Aug04 Quiz-2

1. [Source Q3; MCQ]

Consider the following Vue application with markup "index.html" and JavaScript file "app.js".

index.html:

```
<div id = "app">
    <my-comp>
        <template #header>
            This is header content
        </template>

        <template #footer>
            This is footer content
        </template>

        <p> This is some content </p>
    </my-comp>
</div>
<script src = "app.js"> </script>
```

app.js:

```
Vue.component("myComp", {
    template : `<div>
                    <p>
                        <slot></slot>
                    </p>
                    <p>
                        <slot name="footer"></slot>
                    </p>
                </div>
                `,
})

const app = new Vue({
    el : "#app",
})
```

Suppose you open the "index.html" file in a browser. What will be rendered by the browser?

(a) This is header contentThis is footer contentThis is some content

(b) This is footer contentThis is some content

(c) This is some content

(d) This is some contentThis is header content

(e) This is some contentThis is footer content

[Source ans: e]

2. [Source Q4; MCQ]

Consider the following application with markup index.html and script app.js.

index.html:

```
<body>
    <div id="app">
        <router-view></router-view>
    </div>
</body>
```

app.js:

```
const Header = {
  template: `<div>This is header <router-view /> </div>`,
}
const Error = {
  template: '<div> 404 Page not found</div>',
}
const Profile = {
  template: `<div> Welcome to profile page</div>`,
}
const routes = [
  {
    path: '/',
    component: Header,
    children: [
      { path: 'profile', component: Profile },
      { path: '*', component: Error },
    ],
  },
]
const router = new VueRouter({
  routes,
})
new Vue({
  el: '#app',
```

```
  router,
})
```

Suppose the application is running on http://127.0.0.1:8080. What will be rendered in the browser?

(a) This is header404 Page not found

(b) 404 Page not foundThis is header

(c) Welcome to profile pageThis is header

(d) This is headerWelcome to profile page

[Source ans: a]

3. [Source Q6; MCQ]

Consider the following Vue application with markup "index.html" and JavaScript file "app.js".

index.html:

```
<div id = "app">
    <input v-model = "phrase" @input = "compute_marks">
    <p> {{marks}} </p>
</div>
<script scr = "app.js"></script>
```

app.js:

```
new Vue({
    el : "#app",
    data : {
        phrase : "AppDev",
        marks : 50,
    },

    mounted() {
        this.phrase = "Bravo";
        this.marks = 50;

        if (localStorage.marks) {
            this.phrase += "2";
            this.marks = parseInt(localStorage.marks) + 20;
        }
        else {
            this.phrase += "1";
            this.marks += 10;
        }
    },

    methods : {
        compute_marks() {
            localStorage.setItem("phrase", this.phrase);
            localStorage.setItem("marks", this.marks + 10);
        }
    }
})
```

Suppose you open "index.html" file in a browser, and type the text "Hurray" in the text box shown (after removing the previous text, if any), and hard refresh the page 7 times, without clicking anywhere. What will be the value shown in the text box, and the "marks" placeholder, respectively?

Tip: The "oninput" event occurs immediately after the value of an element has changed.

(a) Bravo1, 70

(b) Bravo2, 90

(c) Bravo1, 60

(d) Bravo2, 70

(e) Bravo1, 80

[Source ans: b]

4. [Source Q8; MCQ]

Consider the following Vue application with markup index.html and JavaScript file app.js.

File: index.html

```
<div id="app"></div>
<script src="script.js"></script>
```

File: script.js

```
new Vue({
    el: '#app',
    template: `<div>
                    Status: {{message}}<br>
                    ETA: {{ETA}}
                </div>`,
    data: {
      message: "",
      ETA: 10,
    },
    beforeCreate() {
        this.message += "Creating, "
        this.ETA -= 1
    },
    created() {
        this.message += "Created, "
        this.ETA -= 1
    },
    beforeMount() {
        this.message += "Mounting, "
        this.ETA -= 1
    },
    mounted() {
        this.message += "Mounted"
        this.ETA -= 1
    },
  })
```

Suppose the application is running on `http://127.0.0.1:8080`. What will be rendered by the browser?

(a) Status: Creating, Created, Mounting, MountedETA: 6

(b) Status: Created, Mounting, MountedETA: 7

(c) Status: Mounting, MountedETA: 8

(d) Status: MountedETA: 9

[Source ans: b]

5. [Source Q10; MCQ]

Consider the following HTML snippet that uses Vue 2 using CDN in the script.js file

Filename: index.html

```
<div id="app">
     <p>Counter: {{ counter }}</p>
     <p>Computed Value: {{ computedValue }}</p>
</div>
```

Filename: script.js

```
new Vue({
    el: "#app",
    data: {
       counter: 0,
    },
    computed: {
      computedValue() {
        return this.counter * 2;
      },
    },
    watch: {
      counter(newVal, oldVal) {
        console.log(`Counter changed from ${oldVal} to ${newVal}`);
      },
    },
    mounted() {
      setInterval(() => {
        this.counter++;
      }, 1000);
    },
});
```

What will be displayed in the console?

(a) Counter changed from 0 to 2, Counter changed from 2 to 4, Counter changedfrom 4 to 6, ...

(b) Counter changed from 0 to 1, Counter changed from 1 to 2, Counter changedfrom 2 to 3, ...

(c) Counter changed from 1 to 3, Counter changed from 3 to 6, Counter changedfrom 6 to 12, ...

(d) error message

[Source ans: b]

6. [Source Q14; MCQ]

Consider the following HTML, with Vue 2 CDN attached to it.

Filename: index.html

```
<div id="app">
  <p>{{ message }}</p>
  <button @click="changeMessage">Change Message</button>
  <button @click="changeCount">Change Count</button>
</div>
```

Filename: script.js

```
<script>
  new Vue({
    el: "#app",
    data: {
      message: "Hello, Vue!",
      count: 0,
    },
    methods: {
      changeMessage() {
        this.message = "Hello, Vue.js!";
      },
      changeCount() {
        this.count++;
      },
    },
    beforeUpdate() {
      console.log("beforeUpdate called");
    },
  });
</script>
```

What will be logged to the console when the Change Count button is clicked?

(a) 'beforeUpdate called'

(b) Nothing will be logged on the console

(c) An error will be thrown

(d) 'beforeUpdate called' will be logged, followed by 'count updated'

[Source ans: b]

7. [Source Q17; MSQ]

Consider the below Vue component template and script definitions that use Vue router.

Vue component template

```
<template>
  <div>
    <h1>{{ pageAction }}</h1>
    <router-link to="/login" v-if="showLogin">Login</router-link>
    <router-link to="/register" v-if="showRegister">Register</router-link>
    <router-view></router-view>
  </div>
</template>
```

Filename: script.js

```
<script>
export default {
  name: 'App',
  data() {
    return {
      pageAction: 'Login or Signup',
      showLogin: true,
      showRegister: false,
    };
  },
  watch: {
    '$route.path'() {
      if (this.$route.path === '/login') {
        this.showRegister = true;
        this.showLogin = false;
        this.pageAction = 'Login Page';
      } else if (this.$route.path === '/register') {
        this.showRegister = false;
        this.showLogin = true;
        this.pageAction = 'Register Page';
      } else {
        this.pageAction = 'Login or Signup';
      }
    },
  },
};
</script>
```

Assuming that the corresponding routes are properly configured, what does this component structure accomplish?

(a) As soon as the app is run by a client, it displays a web page with a heading title"Login or Signup" and two navigation links to "Login" and "Register"

(b) It dynamically updates the heading title based on the route and conditionallyshows navigation links to ”Login” and ”Register”

(c) It has a bug and will throw a runtime error.

(d) The heading title is populated with the value “Login or Signup” when the useropens the app for the first time.

[Source ans: b, d]

2024 Oct27 Quiz-1

8. [Source Q10; MCQ]

Consider the following HTML, with the correct Vue 2 CDN links included.

```
<div id="app">
        <p>Original Order: {{ customerOrder }}</p>
        <p>Updated Order: {{ updatedOrder }}</p>
        <server-component :order="customerOrder"
@order-updated="updateOrder"></server-component>
    </div>

    <script>
        Vue.component('server-component', {
            props: ['order'],
            template: `
          <div>
            <button @click="modifyOrder">Send Updated Order</button>
          </div>
        `,
            methods: {
                modifyOrder() {
                    this.$emit('order-updated', this.order + ' - Extra mayo');
                }
            }
        });
        new Vue({
            el: '#app',
            data() {
                return {
                    customerOrder: 'Burger',
                    updatedOrder: ''
                };
            },
            methods: {
                updateOrder(newOrder) {
                    this.updatedOrder = newOrder;
                }
            }
        });
    </script>
```

What will be displayed in the **<p>** tags after the **button** on the `server-component` is clicked?

(a) Original Order: Burger
Updated Order:

(b) Original Order:
Updated Order:

(c)

Original Order: Burger
Updated Order: - Extra mayo

(d)

Original Order: Burger
Updated Order: Burger - Extra mayo

[Source ans: d]

2024 Dec01 Quiz-2

9. [Source Q5; MCQ]

Consider the following HTML with Vue 2 CDN attached.

```
<div id="app"></div>

<script>
    new Vue({
        el: '#app',
        data() {
            return {
                message: 'Hello'
            };
        },
        beforeCreate() {
            console.log('Before Create: ' + this.message);
            this.message = 'Before Create';
        },
        mounted() {
            this.message = 'Mounted';
            console.log('Mounted: ' + this.message);
        },
        template: `
            <div>
                <p>{{ message }}</p>
            </div>
        `
    });
</script>
```

What will be printed in the browser console ?

(a) 'Before Create: Before Create' and 'Mounted: Hello'

(b) 'Before Create: undefined' and 'Mounted: Hello'

(c) 'Before Create: Hello' and 'Mounted: Mounted'

(d) 'Before Create: undefined' and 'Mounted: Mounted'

[Source ans: d]

10. [Source Q12; MCQ]

Consider the following HTML, which includes Vue 2 and Vue Router 3 CDN links.

```
<body>
    <div id="app"></div>
    <script>
        const Home = { template: `<h1>Home</h1>` }
        const NewestComments = { template: `<h3>Newest Comments</h3>` }
        const TopComments = { template: `<h3>Top Comments</h3>` }
        const SearchResults = { template: `<h3>Search Results</h3>` }
        const TopBar = { template: `<h3>Topbar</h3>` }
        const NavBar = { template: `<h3>Navbar</h3>` }
        const VideoDetail = { template: `<div>Video Details <router-view />
                                        </div>` }

        const routes = [
            { path: '/', component: Home },
            {
                path: '/video/:id',
                component: VideoDetail,
                props: true,
                children: [
                    { path: 'comments/top', component: TopComments },
                    { path: 'comments/newest', component: NewestComments }
                ]
            },
            { path: '/search/:query', component: SearchResults, props: true }
        ];

        new Vue({
            el: '#app',
            template: `<div>
                            <TopBar />
                            <NavBar />
                            <router-view />
                        </div>`,
            router: new VueRouter({ routes }),
            components: { TopBar, NavBar }
        })
    </script>
</body>
```

Given this router setup, which of the following will happen when a user navigates to `localhost:5000/#/video/5/comments/newest`, assuming the webpage is hosted on localhost:5000?

(a)

The `TopNavbar` and `Navbar` components will remain visible, and the `NewestComments` component will be displayed inside the `VideoDetail` component.

(b)

The Home component will be visible.

(c)

The TopComments component will be displayed.

(d)

No component will be shown.

[Source ans: a]

11. [Source Q13; MCQ]

Consider the following HTML, which includes Vue 2 and Vue Router 3 CDN links.

```
<div id="app">
    <player-card>
        <p>Rahul is a versatile player, skilled in defense and attack.</p>
        <template v-slot:extra>
            <p>Position: Captain</p>
            <p>Years on the team: 5</p>
        </template>
    </player-card>
</div>

<script>
    Vue.component('player-card', {
        props: {
            playerName: {
                type: String,
                default: 'Unknown Player'
            }
        },
        template: `
      <div class="card">
        <h2>{{ playerName }}</h2>
        <slot>
          <p>This player is an important part of the team.</p>
        </slot>
        <slot name="extra"></slot>
      </div>
    `
    });

    new Vue({
        el: '#app'
    });
</script>
```

What will be the final-rendered output in the browser?

(a)

Rahul
Rahul is a versatile player, skilled in defense and attack.
Position: Captain
Years on the team: 5

(b)

Rahul
This player is an important part of the team.
Position: Captain
Years on the team: 5

(c)

Unknown Player
Rahul is a versatile player, skilled in defense and attack.
Position: Captain
Years on the team: 5

(d)

Rahul
Position: Captain
Years on the team: 5

[Source ans: c]

12. [Source Q14; MSQ]

Which of the following is/are the use cases of watchers in Vue.js?

(a) To watch a reactive property and perform actions when it changes.

(b) To cache values based on reactive dependencies.

(c) To perform side effects when a specific data property changes.

(d) To create reactive data within the component.

[Source ans: a, c]

2024 Dec22 ET

13. [Source Q5; MCQ]

Consider the below Vue app.

```
new Vue({
  el: '#app',
  data: {
      user: {
      name: 'Abhi',
      age: 30
      }
  },
  watch: {
      user: {
      handler(newValue, oldValue) {
      console.log('User object changed:', newValue);
      },
      deep: true
      }
  }
});
```

What will happen if the following code is executed?

app.user.name = "Dev"

(a) The watcher will not be triggered because "name" is a nested property of"user" object.

(b) The watcher will be triggered and log "User object changed: { name: 'Bob',age: 30 }".

(c) The watcher will be triggered, but it will only log the new "name" value, i.e.,"Dev", and not the entire user object.

(d) The watcher will throw an error because deep watching is not supported onnested objects.

[Source ans: b]

14. [Source Q12; MCQ]

Given this Vue component structure:

```
Vue.component('child', {
    template: `
        <div>
            <slot name="header" :info="info"></slot>
            <slot :info="info"></slot>
        </div>
    `,
    data() {
        return {
            info: { title: 'Hello', desc: 'World' }
        }
    }
})
```

Which slot usage is correct?

(a)
```
<template v-slot:header="data">{{data.title}}</template>
```

(b)
```
<template v-slot:header="{ info }">{{info.title}}</template>
```

(c)
```
<template v-slot:header>{{info.title}}</template>
```

(d)
```
<template v-slot:header=info>{{info.title}}</template>
```

[Source ans: b]

15. [Source Q20; MCQ]

Which of the following statements about the beforeDestroy and destroyed lifecycle hooks in Vue is true?

(a) The “beforeDestroy” hook is called after the component is removed from theDOM, and the “destroyed” hook is called before the component is destroyed.

(b) The “beforeDestroy” hook is called before the component is removed from theDOM, and the “destroyed” hook is called after the component is destroyed.

(c) The “beforeDestroy” hook is called when the component is mounted, and the“destroyed” hook is called before the component is destroyed.

(d) Both hooks are called after the component is destroyed.

[Source ans: b]

16. [Source Q26; MCQ]

Consider the following HTML with Vue CDN included?

```
<div id="app"></div>
    <script>
        Vue.component('my-button', {
            template: `
                <button @click="handleClick">
                <slot></slot>
                </button>
                `,
            methods: {
                handleClick() {
                    console.log('Button')
                    this.$emit('click')
                }
            }
        })

        new Vue({
            el: '#app',
            template: `
            <my-button @click="parentClick">
             Click me
            </my-button>
  `,
            methods: {
                parentClick() {
                    console.log('Parent')
                }
            }
        })

    </script>
```

What will be logged on the console after the button is pressed?

(a) Parent

(b) Button

(c) ButtonParent

(d) ParentButton

[Source ans: c]

17. [Source Q33; MSQ]

Consider the below Vue router configuration.

```
const routes = [
  {
      path: '/user/:id',
      component: UserProfile
  }
];

const router = new VueRouter({
  routes
});
```

Which of the following is a valid way to access the "id" parameter inside the UserProfile component?

(a) this.$route.params.id

(b) this.$router.params.id

(c) this.$route.query.id

(d) this.$route.params['id']

[Source ans: a, d]

2025 Feb23 Quiz-1

18. [Source Q14; MCQ]

Consider the following Vue App and the HTML document given below.

index.html

```
<body>
    <div id="app">
        <my-comp>
            <h3>Exploring Vue JS</h3>
            <template v-slot:ongoing>
                <h3>Learning App Dev 2</h3>
            </template>
        </my-comp>
    </div>
    <script src="./script.js"></script>
</body>
```

script.js

```
const MyComp = {
    name: 'my-comp',
    props: ['tech'],
    template : `<div class="container">
                    <slot name="complete"><h3>Learned DBMS</h3></slot>
                    <slot name="ongoing"></slot>
                    <slot>Exploring Frontend</slot>
                </div>`
}

const app = new Vue({
    el: '#app',
    components: {
        MyComp
    }
})
```

What will be rendered on the browser for code setup given above?

(a) Learning App Dev 2Exploring Vue JS

(b) Learning App Dev 2Exploring Frontend

(c) Learned DBMSLearning App Dev 2 Exploring Frontend

(d) Learned DBMSLearning App Dev 2 Exploring Vue JS

[Source ans: d]

19. [Source Q16; MCQ]

Consider the following HTML with relevant Vue CDN attached.

```
<div id="app">
        <input v-model.number="price" type="number">
        <input v-model.number="quantity" type="number">
        <p>Total: {{ total }}</p>
        <p>Last Updated By: {{ lastUpdatedBy }}</p>
</div>

    <script>
        new Vue({
            el: '#app',
            data: {
                price: 10,
                quantity: 1,
                total: 10,
                lastUpdatedBy: 'initial'
            },
            watch: {
                price: {
                    handler(newVal) {
                        this.total = newVal * this.quantity;
                        this.lastUpdatedBy = 'price';
                    }
                },
                quantity: {
                    handler(newVal) {
                        this.total = this.price * newVal;
                        this.lastUpdatedBy = 'quantity';
                    }
                }
            }
        })
    </script>
```

If the user changes the price to 20 and then immediately changes the quantity to 2, what will be the final values of total and lastUpdatedBy?

(a) total: 20, lastUpdatedBy: ”price”

(b) total: 40, lastUpdatedBy: ”quantity”

(c) total: 20, lastUpdatedBy: ”quantity”

(d) total: 40, lastUpdatedBy: ”price”

[Source ans: b]

2025 Mar16 Quiz-2

20. [Source Q3; MCQ]

What is the correct way to implement two-way data binding in a custom component in Vue.js 2?

(a) Use v-bind directly with a prop

(b) Create a computed property with getter and setter

(c) Use v-model with corresponding prop and emit

(d) Modify the prop value directly in the child component

[Source ans: c]

21. [Source Q8; MCQ]

Consider the below Vue router setup.

```
const Product = {
  template: `<div><h1>Product {{ $route.params.id }}</h1></div>`
};

const ProductReviews = {
  template: `<div><h2>Reviews for Product</h2></div>`
};

const routes = [
  {
    path: '/products/:id',
    component: Product,
    children: [
      {
        path: 'reviews',
        component: ProductReviews
      }
    ]
  }
];

const router = new VueRouter({
  routes
});
```

What will be the behavior when the user visits "/products/123/reviews"?

(a) The Product component will be displayed with the id 123, and theProductReviews component will be displayed as a child route.

(b) The Product component will not display anything because the reviews path isincorrectly nested.

(c) Only the ProductReviews component will be displayed, and the Productcomponent will be ignored.

(d) A 404 error will be shown because the /reviews route cannot be accessed as achild route.

[Source ans: a]

22. [Source Q13; MCQ]

In the Vue application above, what will be the initial rendered message?

Consider the following html with relevant vue cdn links added.

```
<div id="app">
        <async-component ref="child"></async-component>
        <button @click="updateParent">Update Parent</button>
    </div>

    <script>
        Vue.component('async-component', {
            template: '<div>{{message}}</div>',
            data() {
                return {
                    message: 'Initial'
                }
            },
            mounted() {
                setTimeout(() => {
                    this.message = 'Updated in child'
                }, 0)
            }
        })
        new Vue({
            el: '#app',
            mounted() {
                this.$refs.child.message = 'Updated in parent'
            },
            methods: {
                updateParent() {
                    this.$refs.child.message = 'Button clicked'
                }
            }
        })
    </script>
```

(a) ”Initial”

(b) ”Updated in parent”

(c) ”Updated in child”

(d) ”Button clicked”

[Source ans: c]

23. [Source Q17; MCQ]

Consider the below Vue.js application.

```
<template>
  <div>
    <input v-model="name" placeholder="Enter name">
    <input v-model="email" placeholder="Enter email">
    <button @click="submitForm">Submit</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      name: '',
      email: ''
    };
  },
  mounted() {
    if (localStorage.getItem('formData')) {
      const savedData = JSON.parse(localStorage.getItem('formData'));
      this.name = savedData.name;
      this.email = savedData.email;
    }
  },
  watch: {
    name(newVal) {
      localStorage.setItem('formData', JSON.stringify({ name: newVal, email:
localStorage.email }));
    },
    email(newVal) {
      localStorage.setItem('formData', JSON.stringify({ name: localStorage.name,
email: newVal }));
    }
  },
  methods: {
    submitForm() {
```

```
      localStorage.removeItem('formData');
      this.name = '';
      this.email = '';
    }
  }
};
</script>
```

What will happen if the user submits the form and reloads the page?

(a) The form data will persist in the input fields even after submission and pagereload.

(b) The form data will be cleared after submission, and no data will appear after apage reload.

(c) The form data will appear in the inputs after submission and page reload, butonly until the user starts typing again.

(d) The page will go blank.

[Source ans: b]

2025 Apr13 ET

24. [Source Q3; MCQ]

Which of the following correctly describes the lifecycle of a Vue.js component?

(a) beforeCreate -> created -> beforeMount -> mounted -> beforeUpdate ->updated -> beforeUnmount -> unmounted

(b) beforeCreate -> created -> beforeMount -> beforeUpdate -> mounted ->updated -> beforeUnmount -> unmounted

(c) created -> beforeCreate -> beforeMount -> mounted -> updated ->beforeUnmount -> unmounted

(d) created -> beforeCreate -> beforeMount -> mounted -> beforeUnmount ->updated -> unmounted

[Source ans: a]

25. [Source Q11; MCQ]

Consider the below Vue app.

```
<div id="app">
  <router-link to="/foo">Go to Foo</router-link>
  <router-view></router-view>
</div>
<script>
const Foo = { template: '<div>Foo</div>' };
const Bar = { template: '<div>Bar</div>' };

const routes = [
  { path: '/foo', component: Foo },
  { path: '/bar', component: Bar }
  { path: '/', component: Bar }
];

const router = new VueRouter({ routes });

new Vue({
  el: '#app',
  router
});
</script>
```

What happens when the user clicks "Go to Foo"?

(a) Browser navigates to /foo and displays ”Foo”.

(b) Browser navigates to /foo but displays nothing.

(c) Error: No matching route.

(d) None of these

[Source ans: a]

26. [Source Q12; MCQ]

Consider the following Vue component using slots:

```
Vue.component("alert-box", {
  template: `
      <div class="alert-box">
        <strong><slot name="title">Default title</slot></strong>
        <slot>Default content</slot>
      </div>
`,
});
```

How would you use this component to provide custom title and content in the slots?

(a)

```
<alert-box title="Custom Title">Custom content</alert-box>
```

(b)

```
<alert-box>
    <slot name="title">Custom Title</slot>Custom content
</alert-box>
```

(c)

```
<alert-box>
    <template v-slot:title>
       Custom Title
    </template>
    Custom content
</alert-box>
```

(d)

```
<alert-box>Custom Title - Custom content</alert-box>
```

[Source ans: c]

27. [Source Q13; MCQ]

Consider the following Vue component.

```
Vue.component('user-list', {
  data() {
    return {
      users: [
        { id: 1, name: 'Arbok', role: 'Admin' },
        { id: 2, name: 'Bayleaf', role: 'User' },
        { id: 3, name: 'Charzard', role: 'Moderator' }
      ]
    }
  },
  template: `
    <div>
      <slot name="header"></slot>
      <ul>
        <li v-for="user in users" :key="user.id">
          <slot :user="user">
            {{ user.name }}
          </slot>
        </li>
      </ul>
    </div>
  `
})
```

Which of the following is the correct way to use this component with a scoped slot?

(a)

```
<user-list v-slot="{ user }">
     <template>{{ user.name }} ({{ user.role }})</template>
</user-list>
```

(b)

```
<user-list>
     <template v-slot:default="{ user }">
          {{ user.name }} ({{ user.role }})
     </template>
</user-list>
```

(c)

```
<user-list>
      <template scope="{ user }">
            {{ user.name }} ({{ user.role }})
      </template>
</user-list>
```

(d)

```
<user-list>
       <template slot-scope="{ user }">
            {{ user.name }} ({{ user.role }})
       </template>
</user-list>
```

[Source ans: a]

28. [Source Q24; MCQ]

Consider the following Vue App and the HTML document given below.

index.html

```
<div id="app">
     <my-comp>
        <slot>Hello Frontend</slot>
        <slot>Hello VueJS</slot>
     </my-comp>
</div>
<script src="script.js"></script>
```

script.js

```
const myComp = {
    template: `
        <div>
            <slot></slot>
            <slot></slot>
        </div>
    `
}

const app = new Vue({
    components: {
        "my-comp": myComp
    }
}).$mount('#app')
```

Consider the following Vue App and the HTML document given below.
What will be the output on the browser?

(a) Hello Frontend Hello Frontend

(b) Hello VueJS Hello VueJS

(c) Hello Frontend Hello Vue JS

(d) Hello Frontend Hello Vue JS Hello Frontend Hello Vue JS

[Source ans: d]

2025 Aug03 Quiz-2

29. [Source Q7; MCQ]

The following code is using Vue 2 and Vue router 3 running on `http://127.0.0.1:5500`.

Filename: index.html

```
<body>
   <div id="app"></div>
   <script src="app.js" type="module"></script>
</body>
```

Filename: router.js

```
const Home = { template: "<div>Home Page</div>" };
const Dashboard = {
  template: "<div>Dashboard <router-view></router-view></div>",
};
const Stats = { template: "<p>Stats Component</p>" };
const Settings = { template: "<p>Settings Component</p>" };
const NotFound = { template: "<div>404 Not Found</div>" };

const router = new VueRouter({
  routes: [
    { path: "/", component: Home },
    {
      path: "/dashboard",
      component: Dashboard,
      children: [
        { path: "stats", component: Stats },
        { path: "settings", component: Settings },
      ],
    },
    { path: "*", component: NotFound },
  ],
});
export default router;
```

Filename: app.js

```
import router from "./router.js";

new Vue({
  el: "#app",
  router,
  template: `<div><router-view></router-view></div>`,
});
```

What will be displayed when the user visits `http://127.0.0.1:5500/#/dashboard/stats`?

(a) Dashboard only

(b) Stats Component only

(c) DashboardStats Component

(d) 404 Not Found

[Source ans: c]

30. [Source Q8; MCQ]

Consider the following Vue 2 component defined using the CDN version:

Filename: index.html

```
<div id="app">
  <p>{{ message }}</p>
</div>
```

Filename: app.js

```
new Vue({
      el: '#app',
      data() {
        return {
          message: 'Initial'
        };
      },
      created() {
        console.log('Created Hook');
        this.message = 'Created Hook Called';
        this.fetchData();
      },
      mounted() {
        console.log('Mounted Hook');
      },
      methods: {
        async fetchData() {
          const res = await new Promise(resolve => {
            setTimeout(() => resolve('Data Loaded'), 1000);
          });
          this.message = res;
          console.log('Data updated after fetch');
        }
      }
});
```

What will be the sequence of console logs and final content shown on the web page after 1 second?

(a)

Console:

```
Mounted Hook

Created Hook
Data updated after fetch
```

Page: Initial

(b)

Console:

```
Created Hook
Mounted Hook
Data updated after fetch
```

Page: Data Loaded

(c)

Console:

```
Created Hook
Data updated after fetch
Mounted Hook
```

Page: Data Loaded

(d)

Console:

```
Mounted Hook
Data updated after fetch
Created Hook
```

Page: Created Hook Called

[Source ans: b]

31. [Source Q10; MCQ]

Consider the following Vue application with markup "index.html" and JavaScript file "app.js".

Filename: index.html

```
<div id="app"></div>
<script src="app.js"></script>
```

Filename: app.js

```
new Vue({
    el: '#app',
    template: `<div>
                    Phase: {{phase}}<br>
                    Counter: {{counter}}<br>
                    Items: {{items.length}}
                </div>`,
    data: {
        phase: "",
        counter: 0,
        items: []
    },
    beforeCreate() {
        this.phase += "Init-"
        this.counter += 5
        this.items.push("A")
    },
    created() {
        this.phase += "Ready-"
        this.counter += 3
        this.items.push("B")
    },
    beforeMount() {
        this.phase += "Prep-"
        this.counter -= 2
        this.items.push("C")
    },
    mounted() {
        this.phase += "Live"
        this.counter *= 2
        this.items.push("D")
    },
})
```

What will be displayed in the browser when the application loads?

(a)

Phase: Init-Ready-Prep-Live
Counter: 12
Items: 4

(b)

Phase: Ready-Prep-Live
Counter: 2
Items: 3

(c)

Phase: Prep-Live
Counter: -4
Items: 2

(d)

Phase: Live
Counter: 0
Items: 1

(e)

Phase: Ready-Prep-Live
Counter: 12
Items: 3

[Source ans: b]

32. [Source Q12; MSQ]

Consider the below application with markup "index.html" and JavaScript file "app.js".
Filename: index.html

```
<div id="app"></div>
<script src="app.js"></script>
```

Filename: app.js

```
const inventory = {
    101: {product: "Canvas", quantity: 15, warehouse: 3},
    102: {product: "Brushes", quantity: 8, warehouse: 2}
}

const errorPage = {
    template: `<h1>Item not found in inventory!</h1>`
}

const productDetails = {
    template: `<h1>{{item.product}} - {{item.quantity}} units stored
in
                {{item.warehouse}} locations.</h1>`,
    data() {
        return {item: inventory[this.$route.params.productId]}
    },
    watch: {
        '$route'(to, from) {
            this.item = inventory[to.params.productId]
        }
    }
}

const router = new VueRouter({
    mode: 'history',
    routes: [
        { path: '/inventory/:productId', component: productDetails },
        { path: '*', component: errorPage },
    ],
})
```

```
new Vue({
    el: '#app',

    template: '<div><router-view /></div>',
    router,
})
```

Suppose the application is running on "`http://localhost:3000`", select the correct option(s)?

(a)

For the endpoint, `http://localhost:3000/inventory/101`, The browser will render:
Canvas - 15 units stored in 3 locations.

(b)

For the endpoint, `http://localhost:3000/#/inventory/102`, The browser will render:
Brushes - 8 units stored in 2 locations.

(c)

For the endpoint, `http://localhost:3000/inventory`, The browser will render:
Item not found in inventory!

(d)

For the endpoint, `http://localhost:3000/inventory/103`, The browser will render:
Item not found in inventory!

(e)

For the endpoint, `http://localhost:3000/products/101`, The browser will render:
Item not found in inventory!

[Source ans: a, c, e]

33. [Source Q13; MCQ]

The following code is using Vue 2 and Vue router 3 running on `http://127.0.0.1:5500`.

Filename: index.html

```
<body>
   <div id="app"></div>
   <script src="app.js" type="module"></script>
</body>
```

Filename: router.js

```
const Home = { template: "<div>Home Page</div>" };
const Dashboard = {
  template: "<div>Dashboard <router-view></router-view></div>",
};
const Stats = { template: "<p>Stats Component</p>" };
const Settings = { template: "<p>Settings Component</p>" };
const NotFound = { template: "<div>404 Not Found</div>" };

const router = new VueRouter({
  routes: [
    { path: "/", component: Home },
    {
      path: "/dashboard",
      component: Dashboard,
      children: [
        { path: "stats", component: Stats },
        { path: "settings", component: Settings },
      ],
    },
    { path: "*", component: NotFound },
  ],
});
export default router;
```

Filename: app.js

```
import router from "./router.js";

new Vue({
  el: "#app",
  router,
  template: `<div><router-view></router-view></div>`,
});
```

What will be displayed when the user visits the following URLs?

1. `http://127.0.0.1:5500/#/dashboard/unknown`
2. `http://127.0.0.1:5500/#/dashboards/stats`

(a) Both will show Dashboard

(b) 1: Dashboard, 2: 404 Not Found

(c) 1: Dashboard, 2: Stats Component

(d) Both will show 404 Not Found

[Source ans: d]

34. [Source Q14; MCQ]

Consider the following Vue application with markup "index.html" and JavaScript file "app.js".

Filename: index.html

```
<div id="app">
    <data-panel>
        <template #sidebar>
            <span>Navigation Menu</span>
        </template>
        <template #actions>
            <button>Save Changes</button>
            <button>Cancel</button>
        </template>
        <template #default>
            <h3>Main Dashboard</h3>
            <p>Welcome to the control panel</p>
        </template>
        <template #footer>
            <small>Last updated: Today</small>
        </template>
        <div>Extra content outside templates</div>
    </data-panel>
</div>
```

Filename: app.js

```
Vue.component("data-panel", {
    template: `<div class="panel">
                    <header>
                        <slot name="sidebar"></slot>
                    </header>
                    <main>
                        <slot></slot>
                    </main>
                    <section>
                        <slot name="actions"></slot>
```

```
                    </section>
                    <footer>
                        <slot name="summary"></slot>
                    </footer>
                </div>`,
})
const app = new Vue({
    el: "#app",
})
```

What will be the visual output displayed in the browser?

(a)

Navigation Menu

Main Dashboard
Welcome to the control panel
Extra content outside templates

Save Changes Cancel

Last updated: Today

(b)

Navigation Menu

Main Dashboard
Welcome to the control panel
Extra content outside templates

Save Changes Cancel

(c)

Navigation Menu

Extra content outside templates

Save Changes Cancel

(d)

Navigation Menu

Main Dashboard
Welcome to the control panel

Save Changes Cancel
Last updated: Today

[Source ans: b]

2025 Aug31 ET

35. [Source Q26; MCQ]

Consider this Vue.js component code:

```
const TodoApp = {
  template: `
    <div>
      <input v-model="newTodo" @keyup.enter="addTodo" />
      <ul>
        <li v-for="todo in todos" :key="todo.id">{{ todo.text }}</li>
      </ul>
    </div>
  `,
  data() {
    return {
      newTodo: '',
      todos: []
    }
  },
  methods: {
    addTodo() {
      if (this.newTodo) {
        this.todos.push({ id: Date.now(), text: this.newTodo });
        this.newTodo = '';
      }
    }
  }
}
```

What happens when a user types "`Learn Vue`" into the input field and presses Enter?

Note: The `@keyup.enter` directive calls `addTodo()` when the Enter key is pressed.

(a) An error occurs because the `todos` array is empty

(b) The input is cleared, and `Learn Vue` appears in the list

(c) Only the input is cleared, no todo is added

(d) The page refreshes

[Source ans: b]

36. [Source Q27; MCQ]

Given this Vue Router setup:

```
const routes = [
  { path: '/user/:id/profile', component: UserProfile },
  { path: '/user/:id', component: User },
  { path: '*', component: NotFound },
  { path: '/user/:id/settings', component: UserSettings },
];
```

What component renders for URL `/user/123/settings`?

(a) UserProfile

(b) User

(c) NotFound

(d) UserSettings

[Source ans: c]

37. [Source Q30; MSQ]

Assuming that the root Vue instance has already been created and mounted to div app. Which of the following statements are CORRECT?

Consider the following Vue component structure with nested components and complex slot configurations:

```
Vue.component('data-provider', {
    template: `
        <div class="provider">
            <slot name="header" :user="currentUser" :loading="isLoading"></slot>
            <slot name="content" :items="filteredItems" :count="itemCount"></slot>
            <slot :fallback="defaultMessage" :error="errorState"></slot>
        </div>
    `,
    data() {
        return {
            currentUser: { name: 'John', role: 'admin' },
            isLoading: false,
            filteredItems: ['item1', 'item2', 'item3'],
            itemCount: 3,
            defaultMessage: 'No data available',
            errorState: null
        }
    }
});

Vue.component('parent-component', {
    template: `
        <data-provider>
            <template v-slot:header="{ user, loading }">
                <h1 v-if="!loading">Welcome {{ user.name }}</h1>
                <span v-else>Loading...</span>
            </template>

            <template v-slot:content="slotProps">
                <ul>
                    <li v-for="item in slotProps.items" :key="item">
                        {{ item }} ({{ slotProps.count }} total)
                    </li>
                </ul>
            </template>
```

```
            <template v-slot:default="defaultProps">
                <p>{{ defaultProps.fallback }}</p>
            </template>
        </data-provider>
    `
});
```

Now consider this usage of the `parent-component`:

```
<div id="app">
    <parent-component></parent-component>
</div>
```

(a) The header slot will display ”Welcome John” when loading is true

(b) The content slot will render 3 list items with ”item1 (3 total)”, ”item2 (3 total)”,”item3 (3 total)”

(c) The default slot will display ”No data available” from the fallback prop

(d) If we change isLoading: true in data-provider, the header will show”Loading...”

(e) Changing v-slot:default to just v-slot in parent-component would break thefunctionality

[Source ans: b, c, d]

Shared context (source Q31)

Answer the given Subquestions:

38. [Source Q32; MCQ]

Consider the following Vue Router configuration in a Vue 2 application:

```
const routes = [
  {
    path: '/',
    name: 'Home',
    component: HomeView
  },
  {
    path: '/user/:id',
    name: 'UserProfile',
    component: UserProfile
  },
  {
    path: '/product/:category/:id',
    name: 'ProductDetail',
    component: ProductDetail
  }
];
```

Consider the following Vue Router configuration in a Vue 2 application:

In a Vue component, you have these navigation methods:

```vue
<template>
  <div>
    <button @click="goToUser">View User</button>
    <button @click="goToProduct">View Product</button>
    <router-link :to="{ name: 'Home' }">Home</router-link>
  </div>
</template>

<script>
export default {
  methods: {
    goToUser() {
      this.$router.push({ name: 'UserProfile', params: { id: 123 } });
    },
    goToProduct() {
      this.$router.push({ name: 'ProductDetail',
                          params: { category: 'electronics', id: 456}
                        });
    },
    navigateOne() {
      this.$router.push({ name: 'UserProfile', params: { id: 555 },
                          query: { tab: 'settings'}
                        });
    },
    navigateTwo() {
      this.$router.replace({ name: 'Home' });
    },
    goBack() {
      this.$router.go(-1);
    }
}
</script>
```

Consider the following Vue Router configuration in a Vue 2 application:

In a Vue component, you have these navigation methods: What URLs will be generated when the user clicks each navigation element?

(a) User button: /user,Product button: /product, Home link: /home

(b) User button: /user/123,Product button: /product/electronics/456, Home link: /

(c) User button: /UserProfile/123,Product button: /ProductDetail/electronics/456, Home link: /Home

(d) User button: /user?id=123,Product button: /product?category=electronics&id=456, Home link: /

[Source ans: b]

Shared context (source Q32)

Consider the following Vue Router configuration in a Vue 2 application:

```
const routes = [
  {
    path: '/',
    name: 'Home',
    component: HomeView
  },
  {
    path: '/user/:id',
    name: 'UserProfile',
    component: UserProfile
  },
  {
    path: '/product/:category/:id',
    name: 'ProductDetail',
    component: ProductDetail
  }
];
```

Consider the following Vue Router configuration in a Vue 2 application:

In a Vue component, you have these navigation methods:

```
<template>
  <div>
    <button @click="goToUser">View User</button>
    <button @click="goToProduct">View Product</button>
    <router-link :to="{ name: 'Home' }">Home</router-link>
  </div>
</template>

<script>
export default {
  methods: {
    goToUser() {
      this.$router.push({ name: 'UserProfile', params: { id: 123 } });
    },
    goToProduct() {
      this.$router.push({ name: 'ProductDetail',
                          params: { category: 'electronics', id: 456}
                        });
    },
    navigateOne() {
      this.$router.push({ name: 'UserProfile', params: { id: 555 },
                          query: { tab: 'settings'}
                        });
    },
    navigateTwo() {
      this.$router.replace({ name: 'Home' });
    },
    goBack() {
      this.$router.go(-1);
    }
}
</script>
```

Consider the following Vue Router configuration in a Vue 2 application:

In a Vue component, you have these navigation methods: What URLs will be generated when the user clicks each navigation element?

39. [Source Q33; MSQ]

Using the same Vue Router configuration from the previous question, Select ALL correct statements about these navigation methods.

(a) The "View User" link will navigate to /user/789 and match the UserProfile route

(b) The navigateOne() method will create URL /user/555?tab=settings

(c) The $router.replace() method adds a new entry to the browser history

(d) The $router.go(-1) method navigates to the previous page in browser history

[Source ans: b, d]

2025 Oct26 Quiz-1

Shared context (source Q16)

Read the Vue.js code and answer the given subquestions. The parent binds an input with `v-model` and passes the value as a prop to a child component; the child contains a named slot that shows the message in uppercase.

```
<div id="app">
  <p>{{ message }}</p>
  <input v-model="message" />
  <child-component :title="message">
    <template slot="header">
      <h3>{{ message.toUpperCase() }}</h3>
    </template>
  </child-component>
</div>

<script>
Vue.component('child-component', {
  props: ['title'],
  template: `
    <div>
      <h2>{{ title }}</h2>
      <slot name="header"></slot>
      <p>Child Content</p>
    </div>
  `
});

new Vue({
  el: '#app',
  data: {
    message: 'Hello Vue'
  }
});
</script>
```

40. [Source Q17; MCQ]

If the user types "hi" into the input box, what happens on the page, choose the closest to correct?

(a) Only <p> updates to "hi", but <h2> remains "Hello Vue".

(b) Both <p> and <h2> update and display "hi".

(c) Only <h3> changes to "HI", but <p> and <h2> remain unchanged.

(d) Nothing updates at all.

[Source ans: b]

41. [Source Q18; MSQ]

Which Vue.js features are shown in the code?

(a) Two-way data binding using v-model.

(b) Use of a named slot inside a child component.

(c) Passing values from parent to child using props.

(d) Use of lifecycle hooks.

[Source ans: a, b, c]

2025 Dec18 ET

42. [Source Q2; MSQ]

Considering the following code snippet, which statements are true about components in Vue?

```
childComponent.$emit('update', 5)
```

(a) Parent can listen with @update

(b) Parent receives payload via $event

(c) Child can directly modify parent state

(d) $emit triggers a DOM event

[Source ans: a, b]

43. [Source Q16; MCQ]

Consider the following Vue Router setup and HTML.

```
const User = {
  template: `
    <div>
      <h2>User {{ $route.params.id }}</h2>
      <router-view></router-view>
    </div>`
};

const Overview = {
  template: `<p>User Overview</p>`
};

const Orders = {
  template: `<p>User Orders</p>`
};

const routes = [
  {
    path: "/user/:id",
    component: User,
    children: [
      { path: "", component: Overview },
      { path: "orders", component: Orders }
    ]
  }
];

const router = new VueRouter({ routes });
```

```
<!-- html -->
<div id="app">
  <router-view></router-view>
</div>
```

Suppose the webpage is served at http://127.0.0.1:5000. A user visits the URL http://127.0.0.1:5000/#/user/10/orders. What will be rendered on the browser?

(a) User Orders

(b) User 10 User Overview

(c) User 10 User Orders

(d) Nothing is rendered because sibling routes overwrite the parent route.

[Source ans: c]

44. [Source Q22; MCQ]

A Vue Router application uses lazy-loaded route components. When a user navigates to a route with a dynamic parameter (e.g., /product/:id), and the computed property depends on this route parameter, what could be a potential issue if the component is not properly handling asynchronous operations?

(a) The component will throw a critical error and crash the application

(b) The computed property will return undefined because the component hasn't mounted yet

(c) The route parameter will not be accessible in the computed property

(d) The component will display data from the previously loaded route

[Source ans: d]

Shared context (source Q30)

Consider the following Vue 2 (CDN) code. html code:

```
<body>
  <div id="app">
    <h2>Vue Router & Watcher Demo</h2>
    <p>Current userId: {{ userId }}</p>
    <router-link to="/user/1">User 1</router-link> |
    <router-link to="/user/2">User 2</router-link>

    <router-view></router-view>
  </div>
</body>
```

script:

```
<script>
    const User = {
      template: "<div>User component loaded with ID: {{ $route.params.id
}}</div>"
    };

    const routes = [
      { path: '/user/:id', component: User }
    ];

    const router = new VueRouter({ routes });

    new Vue({
      el: '#app',
      router,
      data: {
        userId: null
      },
      watch: {
        '$route.params.id': function(newId) {
          this.userId = newId;
        }
      }
    });
</script>
```

Based on the above data, answer the given subquestions.

45. [Source Q31; MCQ]

What happens when the user clicks on ”User 2” after initially opening ”User 1”?

(a) The route changes to `/user/2` but the `userId` in `data` does not update.

(b) The watcher updates `userId` to `"2"`, and the `<p>` displays it instantly.

(c) The `router-view` still shows User 1 because watchers don’t re-render components.

(d) An error occurs because `$route.params.id` cannot be watched.

[Source ans: b]

46. [Source Q32; MCQ]

Why is the watcher on $route.params.id necessary in this code?

(a) Without it, the userId in data would never change when the route changes.

(b) It is required because Vue Router does not automatically update component data.

(c) It prevents the component from being destroyed and recreated on every navigation.

(d) Both Without it, the userId in data would never change when the route changes and It is required because Vue Router does not automatically update component data are correct.

[Source ans: d]

Shared context (source Q33)

Consider the following Vue.js application with two different route scenarios and a component that uses reactive data binding: Scenario: An e-commerce application where: • Product list component displays products filtered by category • Category changes through router parameters: /products/:category • When a user navigates from /products/electronics to /products/books, the product list should update

Component Code:

```
const ProductList = {
    template: `<div>{{ filteredProducts }}</div>`,
    data() {
        return {
            allProducts: [
                { id: 1, name: 'Laptop', category: 'electronics' },
                { id: 2, name: 'Novel', category: 'books' },
                { id: 3, name: 'Phone', category: 'electronics' }
            ],
            displayedProducts: null
        }
    },
    computed: {
        filteredProducts() {
            return this.allProducts.filter(p => p.category ===
this.$route.params.category);
        }
    },
    watch: {
        '$route.params.category': function(newCategory) {
            this.displayedProducts = this.filteredProducts;
        }
    }
}
```

Based on the above data, answer the given subquestions.

47. [Source Q34; MCQ]

What will be rendered when the user first navigates to /products/electronics?

(a) An empty array because displayedProducts is initialized as null

(b) All three products because the watch handler hasn't been triggered yet

(c) Only the Laptop and Phone (both electronics)

(d) The data will show an error because the category doesn't exist in the array

[Source ans: c]

48. [Source Q35; MCQ]

When the user navigates from /products/electronics to /products/books, what will happen and why?

(a) The computed property will not update because it's being used in a watch handler

(b) The computed property will automatically re-evaluate because the route parameter dependency changed

(c) The displayedProducts variable will update, but the computed property will display stale data

(d) Both the computed property and displayedProducts will update to show books

[Source ans: b]

2026 Mar15 Quiz-1

49. [Source Q12; MCQ]

Consider the following Vue app with html and javascript file given below: File: index.html

```
<div id="app">
  <my-comp>
    <h3>Exploring Vue Js</h3>

    <slot v-slot:last>
      <h3>Learning App Dev 2</h3>
    </slot>
  </my-comp>
</div>
```

File: script.js

```
const MyComp = {
  name: 'my-comp',
  props: ['tech'],
  template: `
    <div class="container">
      <slot name="complete">
        <h3>Learned DBMS</h3>
      </slot>

      <slot>
        Exploring Frontend
      </slot>

      <slot name="last"></slot>
    </div>
  `
};

const app = new Vue({
  el: '#app',
  components: {
    MyComp
  }
});
```

What will be rendered on the browser for the code setup given above ?

(a)

Learned DBMS

Exploring Backend

(b)

Learned DBMS

Exploring Vue Js

This is Last Course

(c)

Learned DBMS

Exploring Vue Js

(d) None of these.

[Source ans: c]

Shared context (source Q15)

Consider the following HTML and Vue component code carefully and answer the given sub-questions. File: index.html

```
<div id="app">
  <p id="text">{{ message }}</p>
</div>
```

File: script.js

```vue
new Vue({
  el: "#app",
  data: {
    message: "Start"
  },
  beforeMount() {
    this.message = this.message + " A";
    console.log("beforeMount message:", this.message);
    },
  mounted() {
    const el = document.getElementById("text");
    if (el) {
      el.innerText = el.innerText + " DOM";
      console.log("DOM text after manipulation:", el.innerText);
    }
  }
});
```

50. [Source Q16; MCQ]

When the `beforeMount()` lifecycle hook is invoked, which of the following statements becomes TRUE?

(a) The `<p id="text">` element exists in the DOM but still contains {{ message }}

(b) The DOM content already reflects the updated value ”Start A”

(c) The `mounted()` hook has already executed

(d) The DOM does not exist at all

[Source ans: a]

51. [Source Q17; MCQ]

What will be the final text content displayed in the browser inside the `<p>` element after the component is fully mounted and all statements in the `mounted()` hook have executed?

(a) `Start A`

(b) `Start A B`

(c) `Start A B DOM`

(d) `Start A DOM`

[Source ans: d]

2026 Apr06 Quiz-2

52. [Source Q3; MCQ]

Consider the following Vue.js application with two different route scenarios and a component that uses reactive data binding: Scenario: An e-commerce application where: Product list component displays products filtered by category• Category changes through router parameters: /products/:category• When a user navigates from /products/electronics to /products/books, the product list should update • Component Code:

```
const ProductList = {
    template: `<div>{{ filteredProducts }}</div>`,
    data() {
        return {
            allProducts: [
                { id: 1, name: 'Laptop', category: 'electronics' },
                { id: 2, name: 'Novel', category: 'books' },
                { id: 3, name: 'Phone', category: 'electronics' }
            ],
            displayedProducts: null
        }
    },
    computed: {
        filteredProducts() {
            return this.allProducts.filter(p => p.category ===
this.$route.params.category);
        }
    },
    watch: {
        '$route.params.category': function(newCategory) {
            this.displayedProducts = this.filteredProducts;
        }
    }
}
```

If the developer changed the template to use displayedProducts instead of filteredProducts, What would be the most likely issue?

(a) Performance would improve because computed properties are slower than data

(b) The initial page load would show an empty list, and only after navigation would data appear correctly

(c) There would be no difference because both will update at the same time

(d) The application would crash because displayedProducts is reactive

[Source ans: b]

53. [Source Q7; MCQ]

In Vue 2 (CDN), which of the following correctly describes the timing of the beforeDestroy and destroyed lifecycle hooks?

(a) beforeDestroy runs after the component has been removed from the DOM, while destroyed runs before destruction.

(b) beforeDestroy runs just before the component is torn down and removed from the DOM, while destroyed runs after the component is fully destroyed.

(c) beforeDestroy runs when the component is first mounted, while destroyed runs before the component is removed.

(d) both hooks run only after the component is destroyed.

[Source ans: b]

54. [Source Q12; MCQ]

Consider the following markup index.html and JavaScript app.js for a Vue application. app.js:

```
const player = {
  template: `<div>
    <h1> Name: {{ name }}</h1>
    <router-view />
  </div>`,
  props: ['name'],
}

const test = {
  template: '<div><h1> Test Runs: {{ runs }} </h1></div>',
  data() {
    return { runs: 5000 }
  },
}

const oneDay = {
  template: '<div><h1> Oneday Runs: {{ runs }} </h1></div>',
  data() {
    return { runs: 10000 }
  },
}

const routes = [
  {
    path: '/player/:name',
    component: player,
    children: [
      { path: '', component: oneDay },
      { path: 'test', component: test },
      { path: 'oneday', component: oneDay },
    ],
```

```
    props: true,
  },
  { path: '*', component: test },
]
const router = new VueRouter({
  routes,
  base: '/',
})

const app = new Vue({
  el: '#app',
  router
})
```

index.html:

```
<div id="app">
   <router-view></router-view>
</div>
<script src="app.js"></script>
```

Suppose the application is running on http://localhost:8080, and the user visit the URL "http://localhost:8080/#/player/rohit/t20". What will be rendered by the browser?

(a) Name: Rohit Oneday Runs: 10000

(b) Test Runs: 5000

(c) Name: Rohit Test Runs: 5000

(d) Oneday Runs: 10000

[Source ans: b]

55. [Source Q13; MCQ]

Consider the following html index.html and JavaScript app.js for a Vue application: File : index.html

```
<div id="app">
  <router-view></router-view>
</div>
```

File : app.js

```
const Waterfall = {
  template: `<p>{{ result }}</p>`,
  data() {
    return {
      districts: [
        { district: "Satara", waterfall: "Vajrai Waterfall" },
        { district: "Shivamogga", waterfall: "Kunchikal Waterfalls" },
        { district: "South Goa", waterfall: "Dudhsagar Falls" },
        { district: "Rewa", waterfall: "Bahuti Falls" }
      ],
      result: ""
    };
  },
  created() {
    const letter = this.$route.params.alpha.toLowerCase();

    for (let i = 0; i < this.districts.length; i++) {
      if (this.districts[i].waterfall[0].toLowerCase() === letter) {
        this.result = this.districts[i].district;
        break;
      }
    }
  }
};

const router = new VueRouter({
  routes: [
    {
      path: "/:alpha",
      component: Waterfall
    }
  ]
});
```

```
new Vue({
  el: "#app",
  router
});
```

Suppose the application is running on http://localhost:8080, and the user visits the URL http://localhost:8080/#/b. What will be rendered by the browser?

(a) Shivamogga

(b) South Goa

(c) Rewa

(d) Satara

[Source ans: c]

56. [Source Q15; MSQ]

Consider the following Vue application: File: Index.html

```
<div id="app">
  <child-comp (Code1)></child-comp>
  <p>{{ message }}</p>
</div>
```

File: app.js

```
Vue.component("child-comp", {
  template: `<button @click="send">Click</button>`,
  methods: {
    send() {
      (Code2)
    }
  }
});

new Vue({
  el: "#app",
  data: {
    message: "Waiting..."
  },
  methods: {
    updateMessage(msg) {
      this.message = msg;
    }
  }
});
```

The application consists of a parent component and a child component. When the button inside the child component is clicked, the parent's message should update and display "Hello" in the browser. Two important lines are missing and marked as (Code1) and (Code2). Which of the following options correctly completes the code?

(a)

```
(Code1) @message="updateMessage('Hello')"
(Code2) this.$emit("message")
```

(b)

```
(Code1) @message="updateMessage"
(Code2) this.$emit("message", "Hello")
```

(c)

```
(Code1) :message="updateMessage"
(Code2) this.emit("message", "Hello")
```

(d)

```
(Code1) @updateMessage="message"
(Code2) this.$emit("updateMessage")
```

[Source ans: a, b]

2026 May06 ET

57. [Source Q9; MCQ]

Consider the below Vue app.

```
<div id="app">
 <router-link to="/about">Go to About</router-link>
 <router-view></router-view>
</div>

<script>
const Home = { template: '<div>Home</div>' };
const About = { template: '<div>About</div>' };

const routes = [
 { path: '/home', component: Home },
 { path: '/about', component: About },
 { path: '*', redirect: '/about' }
];

const router = new VueRouter({ routes });
new Vue({
 el: '#app',
 router
});
</script>
```

What happens when the user navigates to an undefined route like ”/random”?

(a) Browser navigates to /random and shows a blank page.

(b) Browser navigates to /random but renders nothing.

(c) Browser redirects to /about and displays ”About”.

(d) Error: No matching route.

[Source ans: c]

58. [Source Q18; MCQ]

Consider the following Vue application with markup index.html and JavaScript file app.js.

File: index.html

```
<div id="app"></div>
<script src="script.js"></script>
```

File: script.js

```
new Vue({
   el: '#app',
   template: `<div>
                  Status: {{status}}<br>
                  Value: {{value}}
              </div>`,
   data: {
       status: "Initial",
       value: 10,
   },
   beforeCreate() {
       this.status = this.status + " -> Created"
       this.value = this.value + 5
   },
   beforeMount() {
       this.status = this.status + " -> Ready"
       this.value = this.value - 2
   },
   created() {
       this.status = this.status + " -> Mounted"
       this.value = this.value * 2
   },
   mounted() {
       this.status = this.status + " -> Done"
       this.value = this.value / 2
   },
})
```

Suppose the application is running on http://127.0.0.1:8080. What will be rendered by the browser?

(a) Status: Initial -> Created -> Ready -> Mounted -> Done Value: 13

(b) Status: Initial -> Ready -> Mounted -> Done Value: 8

(c) Status: Initial -> Created -> Mounted -> Ready -> Done Value: 14

(d) Status: Initial -> Mounted -> Ready -> Done Value: 9

[Source ans: d]

Shared context (source Q32)

Consider the following Vue 2 application:

```
<div id="app">
   <h3>Selected ID: {{ currentId }}</h3>

   <router-link to="/item/1">Item 1</router-link> |
   <router-link to="/item/2">Item 2</router-link>

   <router-view></router-view>
 </div>

 <script>
   const Item = {
     template: `<p>Item ID from route: {{ $route.params.id }}</p>`,
     updated() {
       console.log("updated hook called");
     }
   };

   const router = new VueRouter({
     routes: [
       { path: "/item/:id", component: Item }
     ]
   });

   new Vue({
     el: "#app",
     router,
     data: {
       currentId: null
     },
     watch: {
       "$route.params.id"(newId) {
         this.currentId = newId;
       }
     }
   });
 </script>
```

Assume the Vue application is loaded at the url

`http://127.0.0.1:5000/#/item/1`

. The user then clicks on the Item 2 router link. Based on the above data, answer the given subquestions.

59. [Source Q33; MCQ]

What happens immediately after clicking "Item 2"?

(a) The Item component is destroyed and recreated, so `updated()` is never called

(b) The route changes, the component is reused, and `updated()` is called

(c) The route changes, but neither watcher nor lifecycle hooks run

(d) A new Vue instance is created for the route

[Source ans: b]

60. [Source Q34; MCQ]

Why is the `updated()` lifecycle hook triggered in this scenario?

(a) Because Vue Router always recreates components on navigation

(b) Because lifecycle hooks run on every URL change regardless of reactivity

(c) Because router-view forces a full re-render of the component tree

(d) Because a watched route parameter updates parent reactive state, triggering a component update

[Source ans: d]

2026 Jul16 Quiz-1

61. [Source Q5; MCQ]

A child component is initially visible using `v-if="show"` . The parent first updates a prop passed to the child, and then sets `show = false` . After the child has already mounted once, which sequence can occur in the child after initial render?

(a) `beforeUpdate` -> `updated` `beforeDestroy` `destroyed`

(b) `beforeDestroy` -> `destroyed` `beforeUpdate` `updated`

(c) `created` -> `mounted` `beforeUpdate` `destroyed`

(d) `updated` -> `beforeCreate` `destroyed` `mounted`

[Source ans: a]

62. [Source Q10; MSQ]

Given the following parent-child setup in Vue 2:

```
<child-box :message="parentMsg" @reply="handleReply"></child-box>
```

and inside the child:

```
this.$emit('reply', 'ok')
```

Which statements are correct?

(a) `message` flows from parent to child through props.

(b) `reply` is a custom event sent from child to parent.

(c) The child should directly change `message` to update the parent source of truth.

(d) If `parentMsg` changes in the parent, the child can receive the updated prop value.

[Source ans: a, b, d]

2026 Aug14 Quiz-2

63. [Source Q10; MSQ]

A Vue 2 component does the following: •loads initial task data from an API, •starts a timer after the component appears on the page, •clears that timer before the component is removed. Which hook choices are appropriate?

(a) `created()` for loading initial data.

(b) `mounted()` for starting DOM-related work or timers.

(c) `beforeDestroy()` for cleanup.

(d) `destroyed()` for declaring reactive `data` .

[Source ans: a, b, c]

64. [Source Q11; MSQ]

In a Vue Router app, a developer uses: • `children` inside a parent route, •

```
{ path: '*', component: NotFound }
```

, • `this.$router.go(-1)` inside a method. Which statements are correct?

(a) `children` can be used to organize nested routes under a parent route.

(b) `*` can act as a fallback for unknown routes.

(c) `this.$router.go(-1)` tries to go back one step in browser history.

(d) Vue Router removes the need for route components.

[Source ans: a, b, c]

65. [Source Q14; MCQ]

Read the following component carefully.

```
<template>
  <div>
    <p>{{ message }}</p>
  </div>
</template>

<script>
export default {
    data() {
       return {
          message: ""
       };
    },
    beforeCreate() {
        console.log("A:", this.message);
    },
    created() {
        console.log("B:", this.message);
        this.message = "Hello from Vue!";
    },
    mounted() {
        console.log("C:", document.querySelector("p").textContent);
    }
};
</script>
```

What will be logged in the console when this component first loads?

(a) `A: "" → B: Hello from Vue! → C: Hello from Vue!`

(b) `A: undefined → B: Hello from Vue! → C: Hello from Vue!`

(c) `A: undefined → B: "" → C: Hello from Vue!`

(d) `A: undefined → B: undefined → C: Hello from Vue!`

(e) `A: "" → B: "" → C: Hello from Vue!`

[Source ans: c]

Shared context (source Q15)

Read the following code carefully and answer the given sub questions: file1: router/index.js

```
const routes = [
  {
    path: "/",
    name: "Home",
    component: HomeComponent
  },
  {
    path: "/profile/:username",
    name: "UserProfile",
    component: ProfileComponent
  },
  {
    path: "/settings",
    name: "Settings",
    component: SettingsComponent
  }
];

const router = new VueRouter({ routes });
```

file2: ProfileComponent.vue

```
<template>
  <div>
    <h1>Welcome, {{ username }}</h1>
    <button @click="goToSettings">Go to Settings</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      username: ""
    };
  },
  created() {
    this.username = _____________________________;  // Line A
  },
  methods: {
    goToSettings() {
      this.$router.push(_____________________________);  // Line B
    }
  }
};
</script>
```

66. [Source Q16; MCQ]

What should be filled in at Line A to correctly read the username from the URL?

(a) `this.$router.params.username`

(b) `this.$route.params.username`

(c) `this.$router.query.username`

(d) `this.$route.query.username`

[Source ans: b]

67. [Source Q17; MCQ]

What should be filled in at Line B to navigate to the Settings page?

(a) `{ path: "Settings" }`

(b) `{ path: "SettingsComponent" }`

(c) `{ name: "Settings" }`

(d)

```
{ name: "SettingsComponent" }
```

[Source ans: c]

2026 Sep13 ET

68. [Source Q4; MCQ]

Consider the following `index.html` and JavaScript `app.js` for a Vue application: File: index.html

```
<div id="app">
  <router-view></router-view>
</div>
```

File: app.js

```
const Flights = {
  template: `<h2>{{ result }}</h2>`,

  data() {
    return {
      routes: [
        { city: "Delhi", connectedTo: ["Mumbai", "Pune"] },
        { city: "Indore", connectedTo: ["Bhopal", "Jaipur"] },
        { city: "Raipur", connectedTo: ["Nagpur", "Hyderabad"] },
        { city: "Rewa", connectedTo: ["Delhi", "Indore", "Raipur"] }
      ],
      result: ""
    };
  },

  created() {
    const destination = this.$route.params.city.toLowerCase();

    for (let i = 0; i < this.routes.length; i++) {
      const connections = this.routes[i].connectedTo;

      for (let j = 0; j < connections.length; j++) {
        if (connections[j].toLowerCase() === destination) {
          this.result = this.routes[i].city;
          break;
        }
      }

      if (this.result) {
        break;
      }
    }
  }
};
```

Suppose the application is running on `http://localhost:8080` , and the user visits the URL:

`http://localhost:8080/#/raipur`

What will be rendered by the browser? [NOTE: `app.js` is divided in two images, consider them as one file content.]

```
const router = new VueRouter({
  routes: [
    {
      path: "/:city",
      component: Flights
    }
  ]
});

new Vue({
  el: "#app",
  router
});
```

(a) Delhi

(b) Indore

(c) Raipur

(d) Rewa

[Source ans: d]

Shared context (source Q28)

Read the following Vue 2 component code carefully (using the Options API via CDN) and answer the given sub questions.

```
<div id="app">
  <p>{{ price }}</p>
  <p>{{ discountedPrice }}</p>
</div>

<script>
new Vue({
  el: "#app",
  data: {
    price: 100,
    discount: 10,
    logCount: 0
  },
  computed: {
    discountedPrice() {
      console.log("computing...");
      return this.price - this.discount;
    }
  },
  watch: {
    price(newVal, oldVal) {
      this.logCount++;
      console.log(`${oldVal} -> ${newVal}`);
    }
  }
});
</script>
```

69. [Source Q30; MCQ]

If `price` is changed twice in two separate updates (100 to 120, then 120 to 150 on a later tick), what is the value of `logCount` afterward?

(a) 0

(b) 1

(c) 2

(d) 3

[Source ans: c]

Shared context (source Q34)

Read the following router and component setup carefully and answer the given sub questions:
Router setup:

```
const router = new VueRouter({
  mode: "hash",
  routes: [
    { path: "/", component: Home },
    { path: "/product/:id", name: "product", component: ProductDetail },
    { path: "/search", name: "search", component: SearchResults }
  ]
});
```

component's method:

```
methods: {
  goToProduct() {
    this.$router.push({ name: "product", params: { id: 42 } });
  },
  goToSearch() {
    this.$router.push({
      name: "search",
      query: { keyword: "shoes", page: 2 }
    });
  }
}
```

70. [Source Q35; MCQ]

What URL does `goToProduct()` navigate the browser to?

(a) /#/product/42

(b) /#/product?id=42

(c) /#/product/:id

(d) /#/42

[Source ans: a]

71. [Source Q36; MCQ]

What URL does `goToSearch()` navigate the browser to?

(a) /#/search/shoes/2

(b) /#/search?keyword=shoes&page=2

(c) /#/search:keyword=shoes:page=2

(d) /#/search

[Source ans: b]

72. [Source Q37; MCQ]

Inside `SearchResults` component, what expression correctly reads the `keyword` value from the URL after navigation?

(a) this.$router.query.keyword

(b) this.$route.query.keyword

(c) this.query.keyword

(d) this.$router.params.keyword

[Source ans: b]